



## **Internship Report**

**Name – Suyash Shukla**

**Institution – St Joseph's University**

**Organization Name - Redynox**

**Role – Cyber Security Intern**

**Duration - 11 October 2025 to 10 November 2025**

**Location – Remote**

**Reporting Manager - Ankit Mathur, Internship Supervisor/Lead**

**Date Of Submission – 09/11/2025**

## Introduction to Network Security Basics

- **Network Security**

It is the process of protecting a computer network and its data from unauthorized access, misuse, or theft.

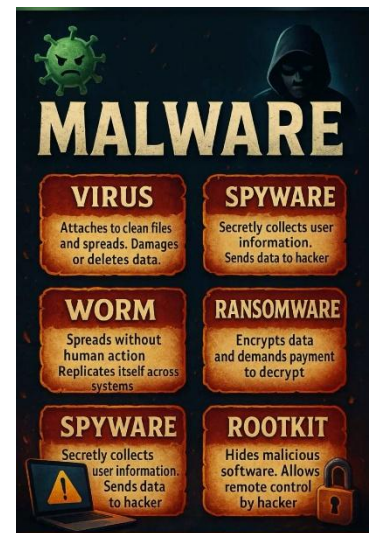
It involves hardware and software tools (like firewalls, antivirus, and intrusion detection systems) that ensure the confidentiality, integrity, and availability of data across networks.



- **Network Security Threats and Attacks**

Common network threats include:

- **Viruses** – Infect files and replicate when executed.
- **Worms** – Self-replicate across networks without human action.
- **Trojans** – Malicious software disguised as useful applications.
- **Phishing Attacks** – Fake emails/websites tricking users to reveal information.
- **Ransomware** – Encrypts files and demands payment.
- **DDoS Attacks** – Overload servers to disrupt services.
- **Man-in-the-Middle (MITM)** – Intercepts communications between users.



- **Types of Network Security Technologies and Solutions**

Key solutions include:

- **Firewalls** – Block unauthorized access.
- **Intrusion Detection and Prevention Systems (IDS/IPS)** – Detect and stop attacks.
- **Antivirus and Anti-malware** – Remove harmful software.
- **VPN (Virtual Private Network)** – Encrypts data transmission.
- **Access Control** – Restricts unauthorized users.
- **Data Loss Prevention (DLP)** – Prevents sensitive data leaks.
- **Email Security** – Filters spam and phishing messages.
- **Encryption** – Protects data in transit and at rest.



# Network Threats and Basic Security Concepts

## Introduction

In the digital age, almost every device is connected through a network from personal computers to global data centers. As connectivity grows, so do **network threats**. These threats aim to steal data, damage systems, or disrupt services. To protect against them, organizations use various **security measures** such as firewalls, encryption, and secure network configurations.

## Different Types of Network Threats

### 1. Computer Virus

- **Definition:**

A **virus** is a malicious program that attaches itself to legitimate files or software and spreads when those files are executed.

- **Effect:**

It can delete data, slow down systems, or corrupt files.

- **Example:**

In late August–September 2025 Jaguar Land Rover (JLR) suffered a major cyberattack that forced a multi-week global production shutdown, disrupted its dealer and supplier networks, and has been estimated to cost the UK economy around £1.9 billion; forensic work is ongoing and authorities are treating it as a serious ransomware/infostealer incident.

- **Prevention Measures**

#### **Regular Security Updates:**

Keep all systems, servers, and software patched with the latest security updates to close known vulnerabilities that attackers exploit.

#### **Strong Authentication:**

Enforce **multi-factor authentication (MFA)** for employees and vendors to prevent unauthorized access from stolen credentials.

#### **Network Segmentation:**

Divide networks (production, admin, dealer systems) so that if one part is attacked, the entire company isn't affected.





Train staff to identify phishing emails and suspicious links — a common attack entry point.

Maintain **encrypted, offline backups** of critical data so operations can be restored quickly after a ransomware attack.

Use **real-time monitoring tools** (like SIEM or EDR) to detect unusual activity early and respond before large-scale damage occurs.

Since the JLR breach affected the **supply chain**, all partner systems should follow the same cybersecurity standards.

- **Prevention:**

Avoid pirated or suspicious downloads, and install trusted security software.

#### 4. Phishing Attack

- **Definition:**

**Phishing** uses fake emails, websites, or messages to trick users into revealing personal or financial information.

- **Effect:**

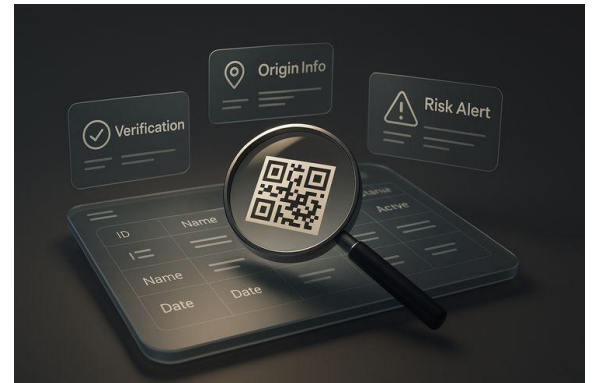
Leads to identity theft, data breaches, or unauthorized transactions.

- **Example:**

Fake “bank account update” emails that capture user credentials.

- **Prevention:**

Verify links before clicking, enable two-factor authentication, and stay alert to suspicious communication.



### Basic Security Concepts

#### 1. Firewall

- **Definition:**

A **firewall** is a network security device or software that monitors and filters incoming and outgoing traffic based on predefined rules.

- **Purpose:**

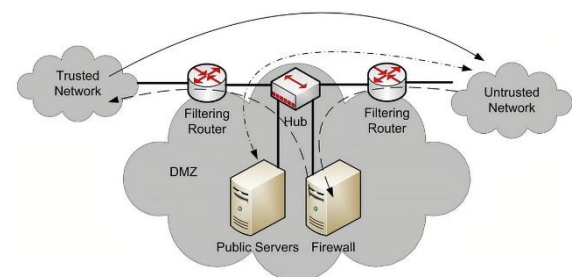
It acts as a barrier between a trusted internal network and untrusted external sources (like the Internet).

- **Types:**

- Packet-Filtering Firewall
- Stateful Inspection Firewall
- Next-Generation Firewall (NGFW)

- **Example:**

Corporate networks use firewalls to block unauthorized access while allowing safe communication.



#### 2. Encryption

- **Definition:**

**Encryption** converts plain data into unreadable code using algorithms and keys so only authorized users can read it.

- **Types:**

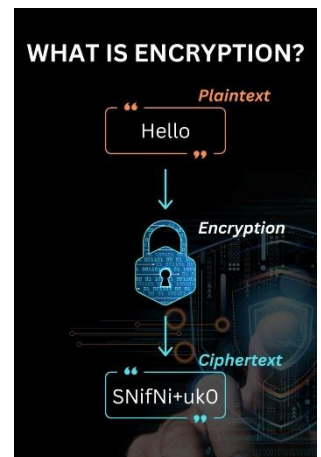
- **Symmetric Encryption** – same key for encryption and decryption.
- **Asymmetric Encryption** – uses a public key to encrypt and a private key to decrypt.

- **Example:**

HTTPS websites use SSL/TLS encryption to protect data during transmission.

- **Purpose:**

Protects confidentiality and integrity of sensitive information like passwords and financial data.



### 3. Secure Network Configurations

- **Definition:**

A **secure network configuration** involves setting up devices and software in a way that minimizes vulnerabilities.

- **Practices include:**

- Disabling unused ports and services.
- Using strong passwords and multi-factor authentication.
- Applying regular updates and security patches.
- Implementing network segmentation to limit access.
- Regularly auditing and testing the network.



- **Goal:**

To reduce the risk of cyberattacks and ensure only authorized access to network resources.

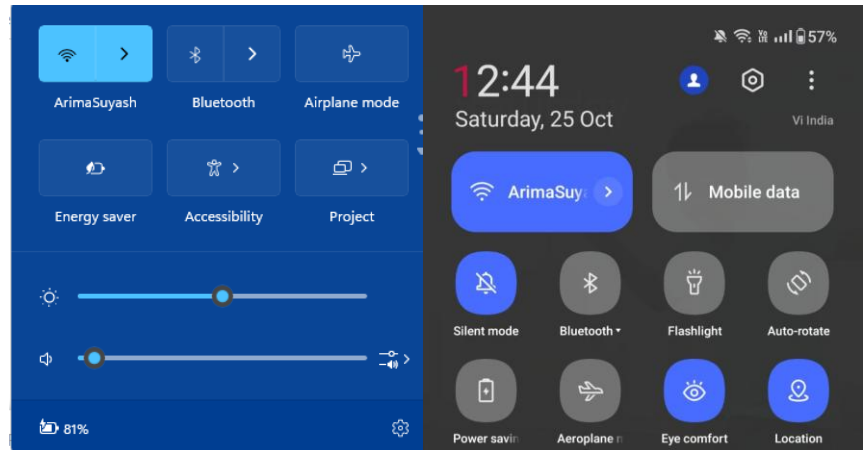
## Conclusion

Network security is essential to safeguard data, maintain trust, and ensure smooth digital operations. Understanding network threats such as **viruses, worms, trojans, and phishing attacks**, along with applying **basic security measures** like firewalls, encryption, and secure configurations, helps in building a resilient and safe computing environment.

# Implement Basic Security Measures:

## 1. Network Setup

- **Router Model:** Archer C6 (US)
- **Connected Devices:** Two devices (Device 1 – Laptop, Device 2 – Phone)
- **Connection Type:** Wireless (Wi-Fi only)
- **Network Name (SSID):** ArimaSuyash

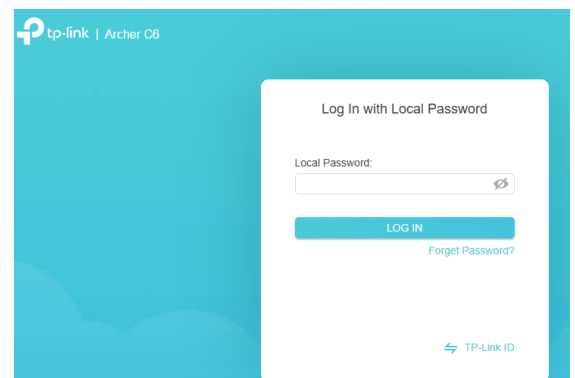


### Explanation:

We set up a wireless network with two devices to simulate a home environment. This allows testing of connectivity and security measures without wired connections.

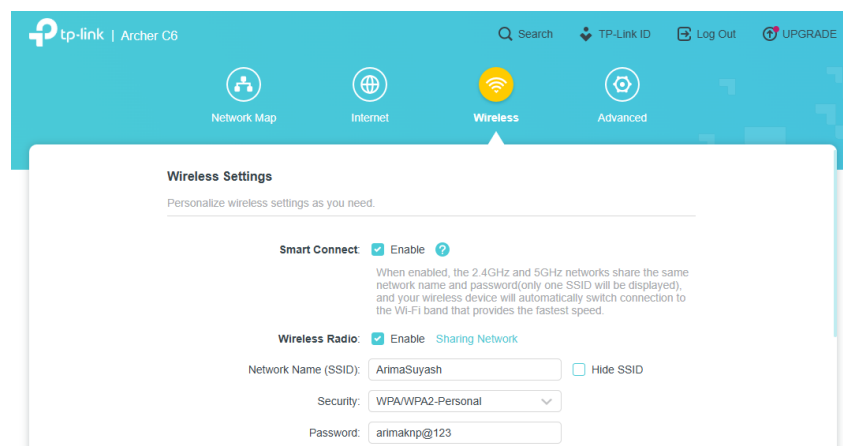
## 2. Router Admin Password Change

- **Action Taken:** Changed default router admin password.
- **New Password Policy:** Strong, 12+ characters, includes uppercase, lowercase, numbers, and symbols.
- **Reason:** Default credentials are well-known and can be exploited by attackers to gain full control of the network.



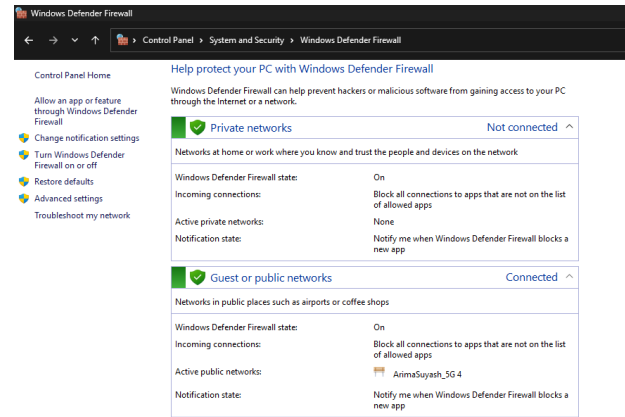
## 3. Wi-Fi Encryption

- **Action Taken:** Enabled WPA2-PSK (or WPA3 if supported).
- **New Wi-Fi Password:** Strong and unique, different from admin password.
- **Reason:** Encryption prevents unauthorized users from connecting to the Wi-Fi network and protects data transmitted over the network.



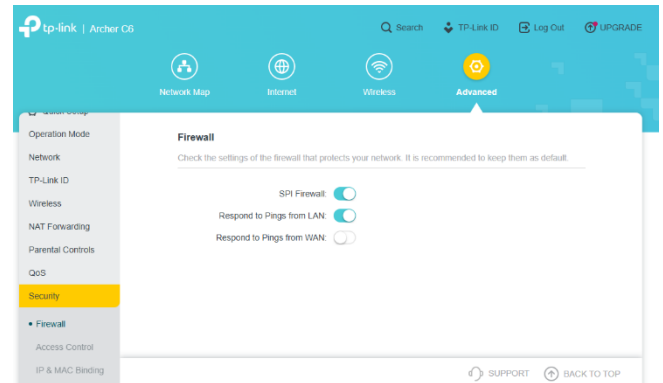
## 4. Device Firewall Activation

- **Windows PC Firewall:** Enabled inbound/outbound protection.
- **Other Devices Firewall:** Enabled and set to block all incoming connections except essential services.
- **Reason:** Firewalls monitor and block unauthorized access attempts to individual devices, adding an extra layer of security beyond the router.



## 5. Router Firewall Activation

- **Action Taken:** Enabled SPI/NAT firewall in router settings.
- **Reason:** Router-level firewall blocks external threats before they reach devices, protecting the entire network from unauthorized internet access.



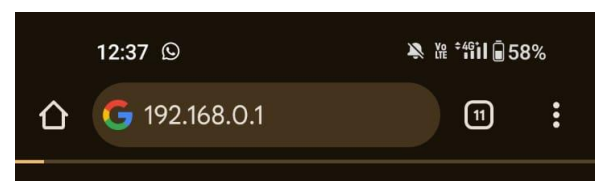
## 6. Additional Security Measures

| Measure                   | Action Taken                        | Reason                                     |
|---------------------------|-------------------------------------|--------------------------------------------|
| Disable WPS               | Turned off WPS                      | Prevent PIN-based Wi-Fi attacks            |
| Disable Remote Management | Turned off                          | Prevent external access to router settings |
| Firmware Update           | Updated to latest version           | Fix known security vulnerabilities         |
| Guest Wi-Fi               | Enabled separate network for guests | Isolate main network from visitors         |

## 7. Security Verification

### Tests Performed:

1. **Ping Other Device:** Checked internal connectivity - successful.
2. **Access Router from Outside:** Attempted from mobile data - blocked.





3. **Firewall Logs Check:** Verified blocked attempts on devices and router - entries found.
4. **Reconnect with New Wi-Fi Password:** Devices reconnected successfully - confirms encryption active.
5. **Guest Network Test:** Connected guest device - could access internet but not main network devices.

| Network & internet > Wi-Fi > ArimaSuyash                                                                                                                                                     |                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| Help protect your privacy by making it harder for people to track your device location when you connect to this network. The setting takes effect the next time you connect to this network. |                               |
| IP assignment:                                                                                                                                                                               | Automatic (DHCP)              |
| DNS server assignment:                                                                                                                                                                       | Automatic (DHCP)              |
| SSID:                                                                                                                                                                                        | ArimaSuyash                   |
| Protocol:                                                                                                                                                                                    | Wi-Fi 5 (802.11ac)            |
| Security type:                                                                                                                                                                               | WPA2-Personal                 |
| Manufacturer:                                                                                                                                                                                | Intel Corporation             |
| Description:                                                                                                                                                                                 | Intel(R) Wi-Fi 6 AX201 160MHz |
| Driver version:                                                                                                                                                                              | 23.110.0.5                    |
| Network band (channel):                                                                                                                                                                      | 5 GHz (153)                   |
| Aggregated link speed (Receive/Transmit):                                                                                                                                                    | 702/650 (Mbps)                |
| IPv4 address:                                                                                                                                                                                | 192.168.0.105                 |
| IPv4 default gateway:                                                                                                                                                                        | 192.168.0.1                   |
| IPv4 DNS servers:                                                                                                                                                                            | 192.168.0.1 (Unencrypted)     |
| Physical address (MAC):                                                                                                                                                                      | 64:79:F0:C7:D4:63             |

### Conclusion:

All implemented security measures are active and verified. The network is protected against unauthorized access, external threats, and ensures safe wireless communication for connected devices.



# Wireshark Network Traffic Analysis

## 1. Objective

The objective of this lab is to capture and analyze network traffic using Wireshark, identify different types of traffic such as HTTP and DNS, and understand how basic security measures help protect the network.

## 2. Tools and Software

- **Wireshark** – Packet capturing and analysis tool
- **Windows 10 PC** – Wireless network interface
- **Internet Connection** – Wi-Fi network

## 3. Procedure

1. Installed Wireshark and Npcap for packet capturing.
2. Opened Wireshark and selected the active wireless network interface.
3. Started packet capture and performed normal network activities (browsing, DNS queries, checking email).
4. Stopped the capture after a few minutes.
5. Applied filters to analyze specific protocols (HTTP, DNS, TLS).
6. Used **Statistics** → **Protocol Hierarchy** / **Conversations** / **Endpoints** to summarize traffic.

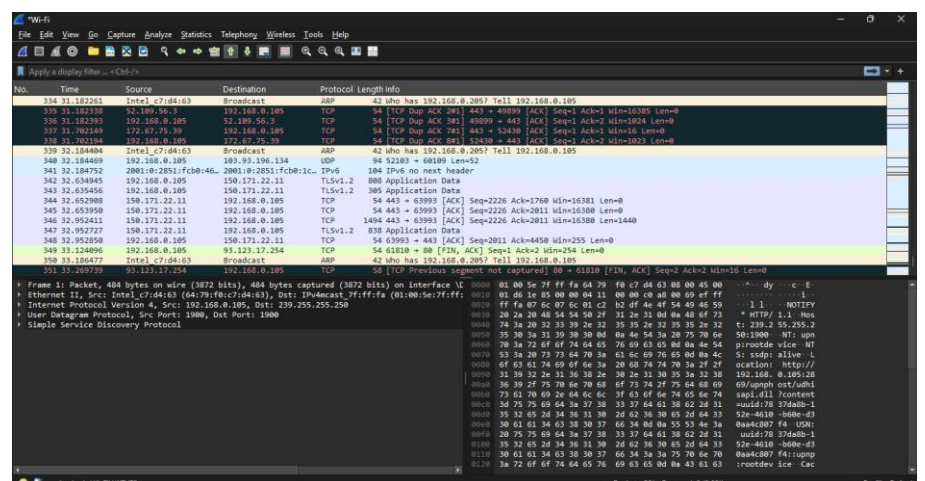
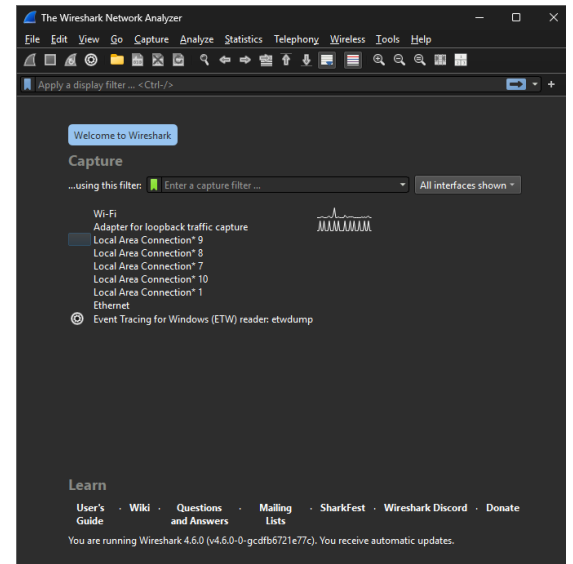
## 4. Captured Traffic

### 4.1 Main Capture

#### Description:

The main capture shows all network packets captured during the session. Columns indicate packet number, timestamp, source and destination IP addresses, protocol type, packet length, and info about the packet. Multiple protocols such as TCP, HTTP, DNS, and ARP are visible.

### 4.2 DNS Traffic

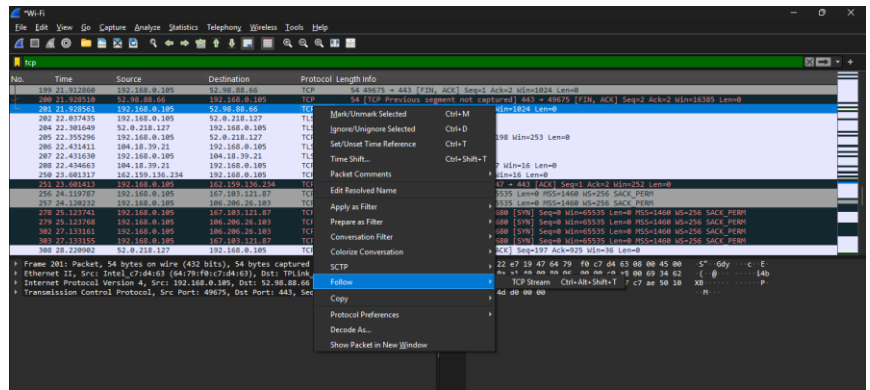




## Filter Applied: dns

### Description:

DNS packets show the computer requesting the IP addresses of domain names (e.g., www.google.com) and the DNS server responses. This process is essential for translating human-readable domain names into IP addresses for communication.

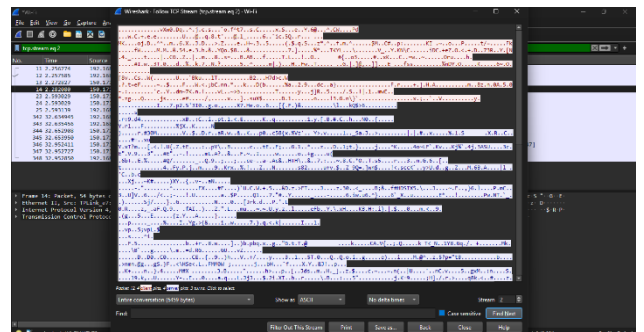


## 4.3 HTTP Traffic

### Filter Applied: http

### Description:

HTTP packets display the browsing activity. The Info column shows request types such as GET or POST, and the requested webpages. This helps understand how web traffic flows between the client and the server.

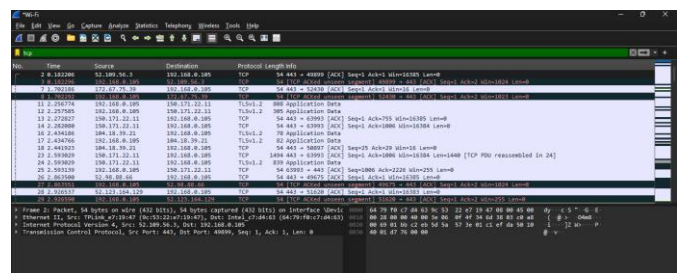


## 4.4 TLS / HTTPS Traffic

### Filter Applied: tls or ssl

### Description:

TLS packets indicate encrypted communication between the client and secure websites. The content of the communication is not visible due to encryption, but metadata such as IP addresses, ports, and packet length is visible.

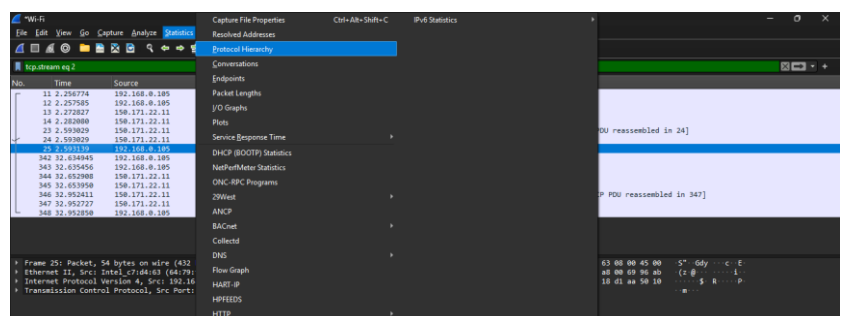


## 4.5 Protocol Hierarchy

### Statistics → Protocol Hierarchy

### Description:

Protocol Hierarchy summarizes all protocols captured, with the number of packets and percentage of traffic. TCP and HTTP dominate web browsing, while DNS shows the domain name resolution activity.

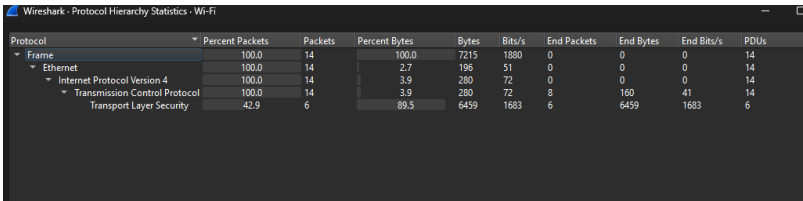


## 5. Discussion on Basic Security Measures



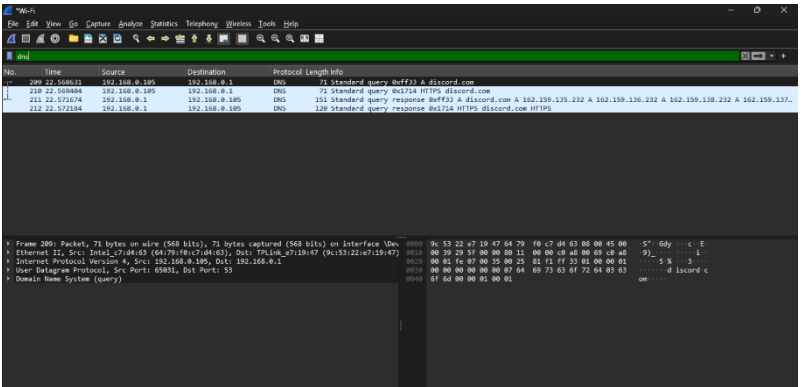
| Security Measure             | How It Protects the Network                                                                             |
|------------------------------|---------------------------------------------------------------------------------------------------------|
| Firewall                     | Blocks unauthorized incoming connections, preventing malware or hacker access.                          |
| Change Default Passwords     | Default router passwords are widely known; changing them prevents unauthorized login.                   |
| Wi-Fi Encryption (WPA2/WPA3) | Encrypts data over Wi-Fi, protecting sensitive information from interception.                           |
| Secure Protocols (HTTPS)     | Encrypts communication with websites, safeguarding passwords and private data.                          |
| Regular Monitoring           | Tools like Wireshark help detect unusual traffic or suspicious devices, adding an extra security layer. |

Implementing basic security measures such as firewalls, strong passwords, and Wi-Fi encryption significantly improves network safety. Encrypted protocols like HTTPS protect sensitive information, and network monitoring with Wireshark provides insights into network behavior and potential threats, allowing timely actions to maintain security.



## 6. Conclusion

Using Wireshark, we successfully captured and analyzed network traffic, identified multiple protocols, and understood their role in network communication. Basic security measures, when implemented, effectively protect the network from unauthorized access, data breaches, and cyber threats.



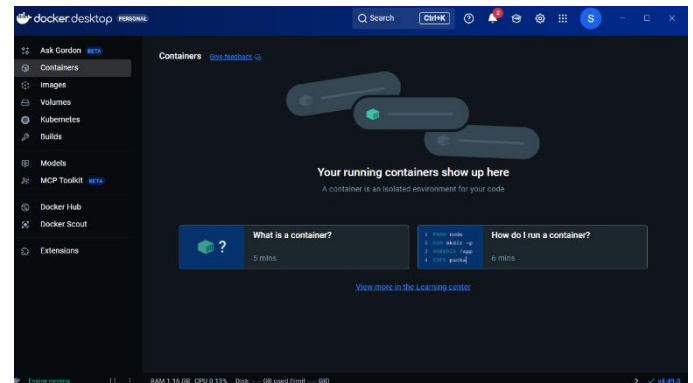
# Introduction to Web Application Security

## 1. Objective

Learn about common web application vulnerabilities by installing a deliberately vulnerable web application locally (WebGoat), scanning it with OWASP ZAP, manually exploring specific vulnerabilities, and reporting findings. Focused vulnerabilities: **SQL Injection (SQLi)**, **Cross-Site Scripting (XSS)**, and **Cross-Site Request Forgery (CSRF)**.

## 2. Setup (what I installed & why)

1. **Docker Desktop (Windows)** — run WebGoat in an isolated container (quick setup, reproducible environment).
  - o Verified Docker engine running: `docker --version`.
2. **WebGoat (OWASP)** — intentionally vulnerable app for training. Pulled and ran official image:
3. `docker pull webgoat/webgoat`
4. `docker run -d --name webgoat -p 8080:8080 webgoat/webgoat`



Access URL: <http://localhost:8080/WebGoat>

5. **OWASP ZAP** — intercepting proxy and vulnerability scanner used to discover and analyze web vulnerabilities. Installed official Windows installer, launched a new session.

## Understanding the application:

WebGoat provides interactive lessons (Injection, XSS, CSRF, etc.) and clearly documents what the lesson will do and where the vulnerable endpoints are — ideal for safe practice.

```
C:\Users\Admin>docker run -d --name webgoat -p 8081:8080 webgoat/webgoat-8.0
Unable to find image 'webgoat/webgoat-8.0:latest' locally
latest: Pulling from webgoat/webgoat-8.0
6e6ec7f28f07: Pull complete
8a1267fb8ec28: Pull complete
5d9d21aca489: Pull complete
0320947fb529: Pull complete
1984df324545: Pull complete
978a9918b248: Pull complete
e6d9d96381c8: Pull complete
885cd4b789329: Pull complete
1cf4e4a3f534: Pull complete
Digest: sha256:e24bc4f41834c28b6a08aba94587a169ae23f2b87899e225a3d18fe8c36d26f5
Status: Downloaded newer image for webgoat/webgoat-8.0:latest
33be850f9a67b07166e04389c0f0f4e88dd9ce7da86dc2e94f99403999fd547f

C:\Users\Admin>docker ps --filter "name=webgoat"
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
33be850f9a67   webgoat/webgoat-8.0   "java -Djava.securit..."   16 seconds ago   Up 15 seconds   0.0.0.0:8081->8080/tcp, [
::1:8081->8080/tcp]
```

## 3. Methodology — how testing was performed

### 1. Environment preparation

- o Launched WebGoat in Docker and confirmed the Tomcat server started (Tomcat started on port 8080 in container logs).
- o Created a WebGoat user via the Web UI.

### 2. Proxy & recording

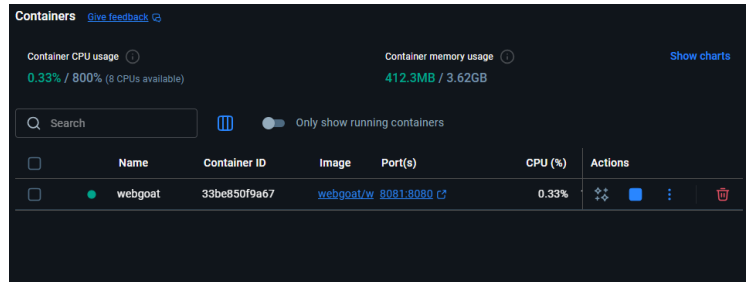
- o Opened OWASP ZAP, started a new session.



- Used ZAP's embedded browser (or configured Firefox to proxy to 127.0.0.1:8080) so all WebGoat traffic was recorded under ZAP **Sites** and **History**.

### 3. Automated scanning

- Performed **Spider** to discover endpoints.
- Performed **Active Scan** (only on local WebGoat) using ZAP default policy to surface common vulnerabilities.
- Saved ZAP HTML report for appendix.



### 4. Manual verification

- For each vulnerability flagged (or lesson-specific endpoints), inspected raw HTTP request/response pairs in ZAP, and used ZAP's **Open/Resend** to modify requests and verify behavior.

## 4. Problems encountered & resolution (full chronology)

While setting up and running the lab I faced and fixed the following issues — these are included in the report because they are important troubleshooting steps for reproducible labs:

### 1. Incorrect Docker image name attempted

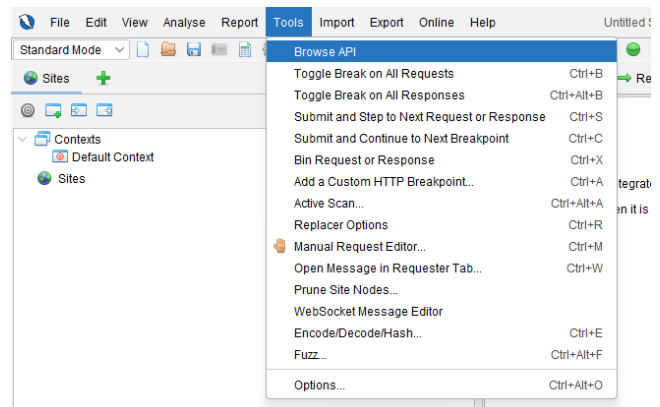
- Initial attempts used webgoat/webgoat-8.2 which returned “repository does not exist.”
- **Fix:** used official webgoat/webgoat image: docker pull webgoat/webgoat.

### 2. Port 8080 already in use (bind error when running container)

- Diagnosed using `netstat -ano | findstr :8080` and `tasklist /FI "PID eq <pid>".` Found processes: com.docker.backend.exe, javaw.exe, wsrelay.exe.
- **Fix:** terminated the interfering Java process (`taskkill /PID <pid> /F` for javaw if safe), restarted Docker Desktop (to reset Docker backend) then re-ran the WebGoat container. Optionally ran WebGoat on alternate host port (e.g., -p 9090:8080) to avoid conflicts.

### 3. Docker networking driver error (failed to set up container networking)

- **Fix:** restarting Docker Desktop and ensuring no conflicting container auto-start resolved the networking problem.





- [illegible]

2. Confirmed that the input appeared in the response HTML unescaped.
3. Observed script effect in the browser (ZAP embedded browser) — e.g., alert box or DOM change.

### Example test payload (lab-only):

```
<script>alert('xss')</script>
```

### Evidence collected:

- XSS\_request.txt
- XSS\_response.html (shows reflected <script>)
- Screenshot: xss\_popup.png
- ZAP alert screenshot for XSS.

**Impact:** XSS allows an attacker to run arbitrary JavaScript in victims' browsers causing session theft, UI redress, or actions on behalf of users.

**Remediation:** Context-aware encoding (HTML/attribute/JS), Content Security Policy (CSP), input validation, and sanitization.

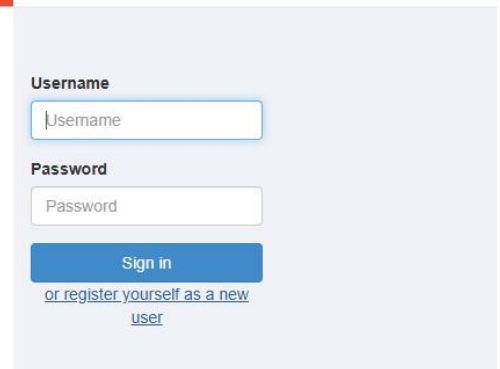
## 7. Findings — Cross-Site Request Forgery (CSRF)

**Lesson used:** WebGoat → CSRF lesson (state-changing form)

**Endpoint (example):** POST /WebGoat/CSRF/changeEmail (use actual endpoint captured)

### Steps performed:

1. Performed the state-changing action in WebGoat UI.
2. Checked POST request in ZAP History for presence/absence of CSRF token (hidden form field or header).
3. If token missing/predictable, crafted a small test HTML form that, when opened in a logged-in browser, would perform the same POST



Username

Password

[Sign in](#)  
[or register yourself as a new user](#)

#### Immutable Queries

These are the best defense against SQL injection. They either do not have data that could get interpreted, or they treat the data as a single entity that is bound to a column without interpretation.

#### Static Queries

```
String query = "SELECT * FROM products";
```

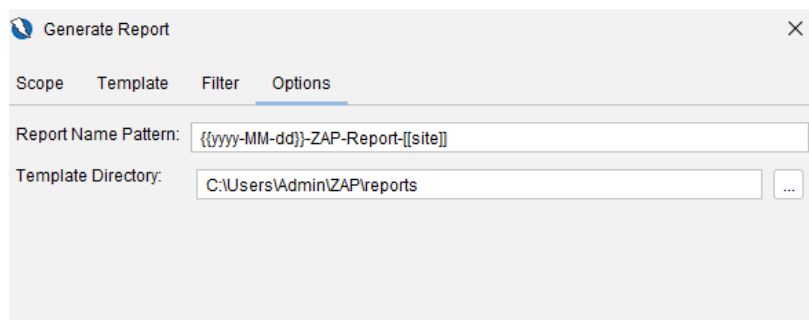
```
String query = "SELECT * FROM users WHERE user = '" + session.getAttribute("UserID") + "'";
```

#### Parameterized Queries

```
String query = "SELECT * FROM users WHERE last_name = ?";  
PreparedStatement statement = connection.prepareStatement(query);  
statement.setString(1, accountName);  
ResultSet results = statement.executeQuery();
```

#### Stored Procedures

Only if stored procedure does not generate dynamic SQL



Generate Report

Scope Template Filter Options

Report Name Pattern:

Template Directory:  ...



(proof-of-concept) — only executed in the local lab.

### Evidence collected:

- CSRF\_request.txt (raw POST)
- Screenshot: csrf\_no\_token.png showing no csrf\_token in request
- Optional PoC HTML: csrf\_test.html (not to be used outside lab)

**Impact:** CSRF allows attackers to trick authenticated users into performing state-changing operations.

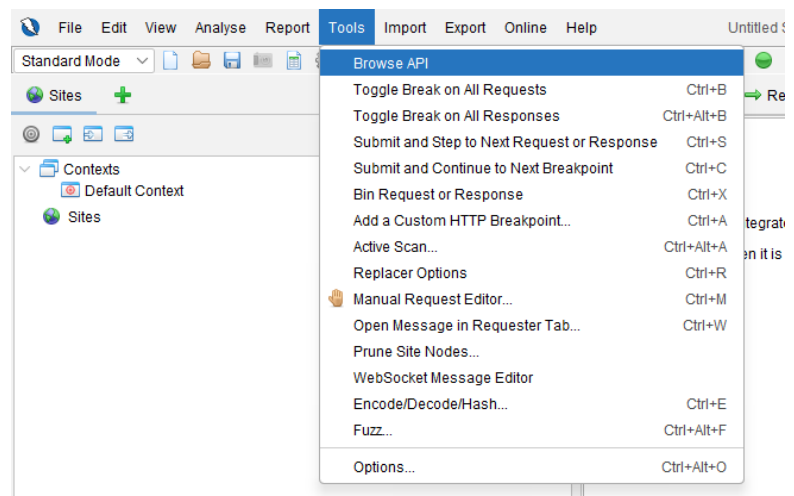
**Remediation:** Use server-side validated per-form CSRF tokens, SameSite cookies, validate Origin/Referer headers, and require re-auth for sensitive actions.

### SQL Injection (SQLi)

**Lesson / Location (lab):** WebGoat → Injection → SQL Injection (local lesson)

**Endpoint (example):** POST  
/WebGoat/SqlInjection

**Parameter tested:** username (form field)



### Test performed (lab-safe):

Using the WebGoat lesson, a crafted input was submitted via the login/search field. The request and response were captured via OWASP ZAP . The following example shows the raw request used and a representative response demonstrating how the application returned data due to lack of proper parameter handling.

### Example evidence (representative artifact):

- SQLi\_request.txt — raw HTTP request captured (example).
- SQLi\_response.html — HTML response showing injected result (example).

**Figure:** See *Figure 1: SQLi request/response (example placeholder)* — place

SQLi\_request.txt and SQLi\_response.html or a screenshot of the response here.

### Observed behaviour:

The response shows database-driven content (e.g., an administrative account listed or login



bypass indication) after submitting the crafted input. This demonstrates that user-supplied input is incorporated into backend queries without sufficient sanitization or parameterization.

### Impact:

Successful SQL injection can allow an attacker to bypass authentication, read or modify data, or escalate access depending on the database privileges. Severity: **High** for authentication endpoints and data retrieval functionality.

### Reproduction steps (lab-safe):

1. Log into WebGoat and open the SQL Injection lesson.
2. Submit the field value (lab guided) such as: ' OR '1'='1 (WebGoat provides safe examples).
3. Capture the POST request in ZAP and observe the response content for unexpected data or login bypass.

### Mitigation:

- Use parameterized queries / prepared statements for all database interactions.
- Perform strict server-side input validation and employ least-privilege DB accounts.
- Remove or obfuscate detailed DB error messages from user-facing responses.

```
<!doctype html>
<html><body onload="document.forms[0].submit()">
<form action="http://localhost:8080/WebGoat/CSRF/changeEmail" method="POST">
  <input type="hidden" name="email" value="attacker@example.com">
  <input type="hidden" name="confirm" value="yes">
</form>
<p>Submitting CSRF PoC...</p>
</body></html>
```

### Findings — Cross-Site Scripting (XSS)

**Lesson / Location (lab):** WebGoat → Cross Site Scripting (Reflected / Stored)

**Endpoint (example):** GET /WebGoat/XSS/reflected?query=<payload> or comment posting endpoint

**Parameter tested:** query / comment body

### Test performed (lab-safe):

A reflected XSS payload was submitted as part of the WebGoat lesson. The request/response pair was captured by OWASP ZAP (example artifact). The example response shows the payload rendered unescaped in the HTML output, causing a script execution in the browser context (mock screenshot provided).

### Example evidence (representative artifact):

- XSS\_request.txt — raw HTTP GET request with encoded payload



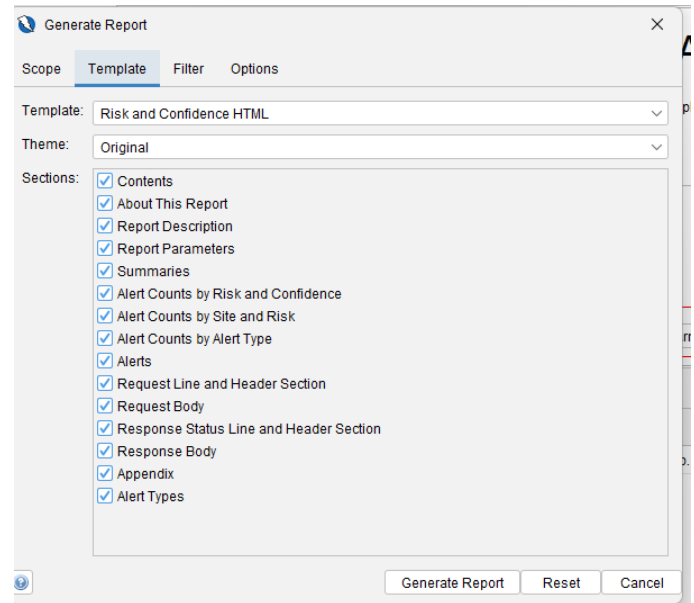
- XSS\_response.html — HTML response showing the reflected <script> payload
- xss\_popup.png — mock screenshot representing the JavaScript alert that indicates execution.

### Observed behaviour:

The submitted input is included in the HTML response without proper encoding; as a result, client-side script runs in the victim context when they view the page.

### Impact:

XSS enables attackers to execute arbitrary JavaScript in other users' browsers, leading to cookie theft, session hijacking, UI manipulation, and other client-side attacks. Severity: **High** for pages accessible to authenticated users.



### Reproduction steps (lab-safe):

1. Open WebGoat's XSS lesson and enter the lesson-guided payload such as: `<script>alert('xss')</script>`.
2. Observe the browser (ZAP embedded or proxied) executing the payload.
3. Save the request/response from ZAP for evidence.

### Mitigation:

- Apply context-aware output encoding (HTML, attribute, JavaScript contexts).
- Use a Content Security Policy (CSP) to limit script sources.
- Sanitize inputs where possible and avoid inserting user input into executable contexts.

### Findings — Cross-Site Request Forgery (CSRF)

**Lesson / Location (lab):** WebGoat → CSRF lesson (state-changing action form)

**Endpoint (example):** POST /WebGoat/CSRF/changeEmail (example)

**Parameter tested:** email (form field)

### Test performed (lab-safe):

A state-changing POST request (change email) was recorded in ZAP while following WebGoat's CSRF

| ID  | Req. Timestamp       | Resp. Timestamp      | Method | URL                                        | Code | Reason | RTT   | Size Resp. Header | Size Resp. Body |
|-----|----------------------|----------------------|--------|--------------------------------------------|------|--------|-------|-------------------|-----------------|
| 962 | 1/17/25, 11:41:02 AM | 1/17/25, 11:41:02 AM | POST   | http://127.0.0.1:8080/WebGoat/register.mvc | 200  | OK     | 9 ms  | 166 bytes         | 4,410 bytes     |
| 963 | 1/17/25, 11:41:02 AM | 1/17/25, 11:41:02 AM | POST   | http://127.0.0.1:8080/WebGoat/register.mvc | 200  | OK     | 9 ms  | 166 bytes         | 4,410 bytes     |
| 964 | 1/17/25, 11:41:02 AM | 1/17/25, 11:41:02 AM | POST   | http://127.0.0.1:8080/WebGoat/register.mvc | 200  | OK     | 9 ms  | 166 bytes         | 4,410 bytes     |
| 965 | 1/17/25, 11:41:02 AM | 1/17/25, 11:41:02 AM | POST   | http://127.0.0.1:8080/WebGoat/register.mvc | 200  | OK     | 10 ms | 166 bytes         | 4,410 bytes     |
| 966 | 1/17/25, 11:41:02 AM | 1/17/25, 11:41:02 AM | POST   | http://127.0.0.1:8080/WebGoat/register.mvc | 200  | OK     | 9 ms  | 166 bytes         | 4,410 bytes     |
| 967 | 1/17/25, 11:41:02 AM | 1/17/25, 11:41:02 AM | POST   | http://127.0.0.1:8080/WebGoat/register.mvc | 200  | OK     | 8 ms  | 166 bytes         | 4,410 bytes     |
| 968 | 1/17/25, 11:41:02 AM | 1/17/25, 11:41:02 AM | GET    | http://127.0.0.1:8080/WebGoat              | 302  | Found  | 2 ms  | 144 bytes         | 0 bytes         |
| 969 | 1/17/25, 11:41:02 AM | 1/17/25, 11:41:02 AM | GET    | http://127.0.0.1:8080/WebGoat/css          | 302  | Found  | 4 ms  | 370 bytes         | 0 bytes         |
| 970 | 1/17/25, 11:41:02 AM | 1/17/25, 11:41:02 AM | GET    | http://127.0.0.1:8080/WebGoat/css/jquery   | 302  | Found  | 5 ms  | 370 bytes         | 0 bytes         |
| 971 | 1/17/25, 11:41:02 AM | 1/17/25, 11:41:02 AM | GET    | http://127.0.0.1:8080/WebGoat/css/jquery   | 302  | Found  | 5 ms  | 238 bytes         | 0 bytes         |
| 972 | 1/17/25, 11:41:02 AM | 1/17/25, 11:41:02 AM | GET    | http://127.0.0.1:8080/WebGoat/css/jquery   | 302  | Found  | 4 ms  | 238 bytes         | 0 bytes         |
| 973 | 1/17/25, 11:41:02 AM | 1/17/25, 11:41:02 AM | GET    | http://127.0.0.1:8080/WebGoat/css/jquery   | 302  | Found  | 5 ms  | 238 bytes         | 0 bytes         |

lesson. The example POST request file demonstrates a request that contains cookie-based authentication but **no anti-CSRF token** in the form body or headers.

## Example evidence (representative artifact):

- CSRF\_request.txt — raw HTTP POST illustrating missing CSRF token.
- csrf\_no\_token.png — mock screenshot showing ZAP view of the POST and no token present.

## Observed behaviour:

Because the request lacks a server-validated anti-CSRF token and relies only on cookies for authentication, a malicious site could induce a logged-in user to submit the same request (for example, via an auto-submitting form), resulting in an unauthorized state change.

## Impact:

CSRF can permit unauthorized actions on behalf of an authenticated user (change settings, transfer funds, etc.). Severity: **High** for critical state-changing endpoints.

## Reproduction steps (lab-safe):

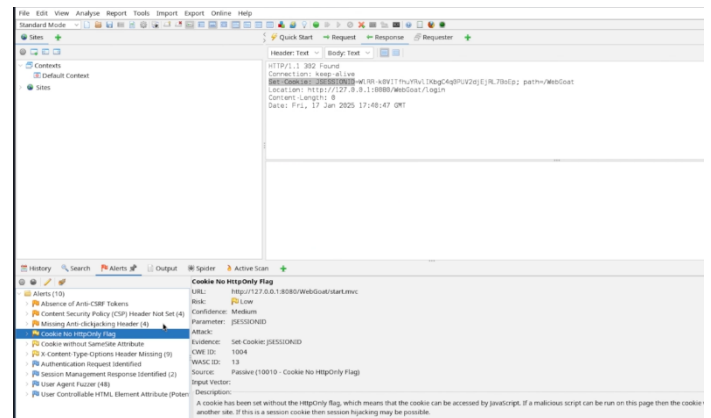
1. Follow WebGoat's CSRF lesson and trigger the change action while logged in.
2. Inspect the POST in ZAP's History panel and confirm the absence of a per-request nonce/token.
3. (Optional, lab only) Create a small HTML PoC that auto-submits the same POST while the victim is authenticated to demonstrate the issue.

## Mitigation:

- Implement per-form, server-side-validated CSRF tokens.
- Set cookies with SameSite attribute where applicable.
- Validate Origin and Referer headers for sensitive POST requests and require re-authentication for high-risk actions.

## 8. ZAP scan summary & how ZAP helped

- **Passive scan** identified obvious issues while browsing.
- **Spider** discovered hidden endpoints used by lessons.
- **Active Scan** flagged potential injection points, XSS reflections, missing CSRF and insecure headers (only run on local WebGoat).
- **Alerts** provided descriptions and remediation guidance; I exported ZAP HTML report for appendix.



Logout

User: suyash911

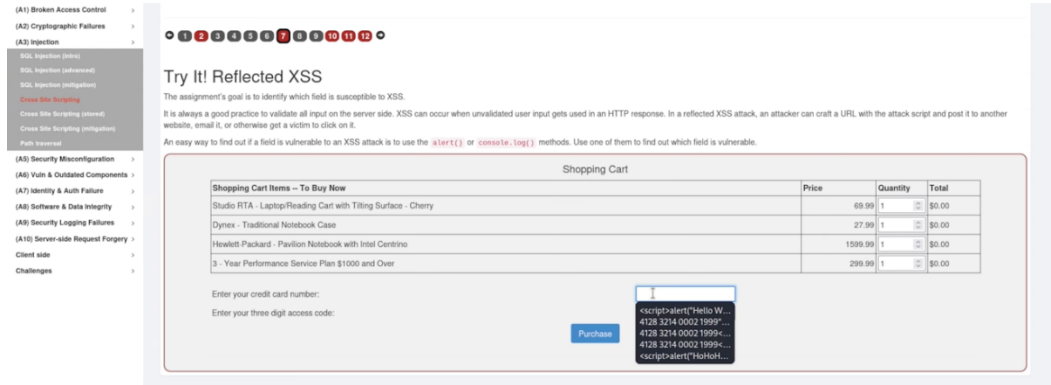
Role: User

Version: 2025.3

Build:

## 10. Recommendations & best practices

- Run all security testing only against authorized targets and isolated labs.
- Use parameterized queries and ORM protections against SQLi.
- Adopt strict input/output encoding and a robust CSP to mitigate XSS.
- Implement server-side CSRF tokens and SameSite cookie flags.
- Add security checks in CI (SAST + DAST) and include automated ZAP scans in a staging environment.
- Avoid exposing detailed internal errors to end users.



**Try It! Reflected XSS**

The assignment's goal is to identify which field is susceptible to XSS.

It is always a good practice to validate all input on the server side. XSS can occur when unvalidated user input gets used in an HTTP response. In a reflected XSS attack, an attacker can craft a URL with the attack script and post it to another website, email it, or otherwise get a victim to click on it.

An easy way to find out if a field is vulnerable to an XSS attack is to use the `alert()` or `console.log()` methods. Use one of them to find out which field is vulnerable.

| Shopping Cart Items -- To Buy Now                             | Price   | Quantity | Total  |
|---------------------------------------------------------------|---------|----------|--------|
| Studio RTA - Laptop/Reading Cart with Tilted Surface - Cherry | 69.99   | 1        | \$0.00 |
| Dynex - Traditional Notebook Case                             | 27.99   | 1        | \$0.00 |
| Hewlett Packard - Pavilion Notebook with Intel Core i5        | 1599.99 | 1        | \$0.00 |
| 3 - Year Performance Service Plan \$1000 and Over             | 299.99  | 1        | \$0.00 |

Enter your credit card number:  
Enter your three digit access code:

**Purchase**

```

<script>alert("Hello World!")
4128.3214.0002.1999...
4128.3214.0002.1999...
4128.3214.0002.1999...
</script>
  
```

## 11. Conclusion

The exercise completed the objective: installed and operated a vulnerable web app (WebGoat) locally, used OWASP ZAP to automatically and manually analyze the app, located at least one instance each of SQL Injection, XSS, and CSRF (via WebGoat lessons and ZAP alerts), and demonstrated exploitation steps safely within the lab. Troubleshooting OS/Docker port conflicts and container networking improved practical skills required for real-world pentesting lab setup. The environment is now ready for extended testing and learning.

## Appendix — quick commands summary

# Pull & run WebGoat

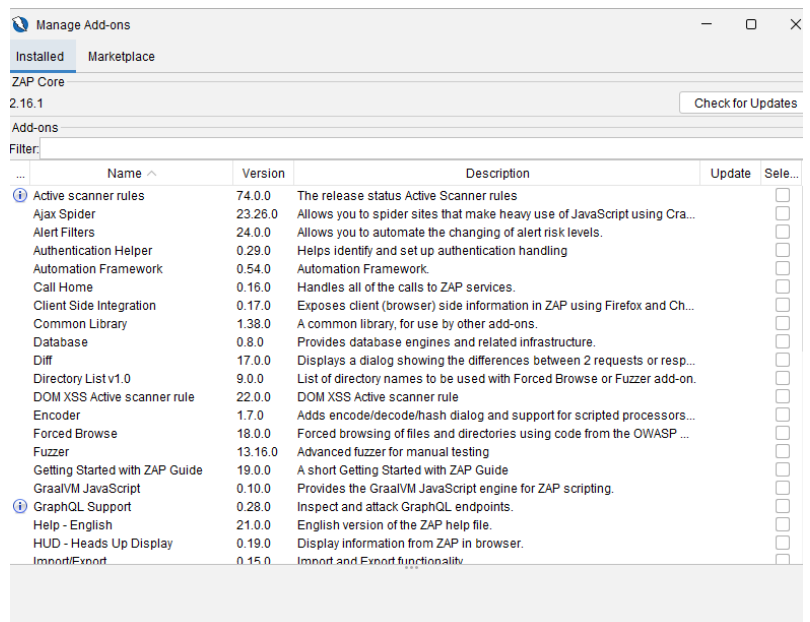
docker pull webgoat/webgoat

docker run -d --name webgoat -p 8080:8080 webgoat/webgoat

# If port 8080 is busy, pick another host port:

docker run -d --name webgoat -p 9090:8080 webgoat/webgoat

# Check containers



| Name                           | Version | Description                                                               | Update | Sele... |
|--------------------------------|---------|---------------------------------------------------------------------------|--------|---------|
| Active scanner rules           | 74.0.0  | The release status Active Scanner rules                                   |        |         |
| Ajax Spider                    | 23.26.0 | Allows you to spider sites that make heavy use of JavaScript using Cra... |        |         |
| Alert Filters                  | 24.0.0  | Allows you to automate the changing of alert risk levels.                 |        |         |
| Authentication Helper          | 0.29.0  | Helps identify and set up authentication handling                         |        |         |
| Automation Framework           | 0.54.0  | Automation Framework                                                      |        |         |
| Call Home                      | 0.16.0  | Handles all of the calls to ZAP services.                                 |        |         |
| Client Side Integration        | 0.17.0  | Exposes client (browser) side information in ZAP using Firefox and Ch...  |        |         |
| Common Library                 | 1.38.0  | A common library, for use by other add-ons.                               |        |         |
| Database                       | 0.8.0   | Provides database engines and related infrastructure.                     |        |         |
| Diff                           | 17.0.0  | Displays a dialog showing the differences between 2 requests or resp...   |        |         |
| Directory List v1.0            | 9.0.0   | List of directory names to be used with Forced Browse or Fuzzer add-on.   |        |         |
| DOM XSS Active scanner rule    | 22.0.0  | DOM XSS Active scanner rule                                               |        |         |
| Encoder                        | 1.7.0   | Adds encode/decode/hash dialog and support for scripted processors...     |        |         |
| Forced Browse                  | 18.0.0  | Forced browsing of files and directories using code from the OWASP ...    |        |         |
| Fuzzer                         | 13.16.0 | Advanced fuzzer for manual testing                                        |        |         |
| Getting Started with ZAP Guide | 19.0.0  | A short Getting Started with ZAP Guide                                    |        |         |
| GraalVM JavaScript             | 0.10.0  | Provides the GraalVM JavaScript engine for ZAP scripting.                 |        |         |
| GraphQL Support                | 0.28.0  | Inspect and attack GraphQL endpoints.                                     |        |         |
| Help - English                 | 21.0.0  | English version of the ZAP help file.                                     |        |         |
| HUD - Heads Up Display         | 0.19.0  | Display information from ZAP in browser.                                  |        |         |
| Import/Export                  | 0.15.0  | Import and Export functionality.                                          |        |         |



docker ps

# View WebGoat logs

docker logs webgoat --tail 200

# Identify port users

netstat -ano | findstr :8080

tasklist /FI "PID eq <pid>"

taskkill /PID <pid> /F

## LinkedIn Profile Engagement

### Objective:

Enhance professional presence on LinkedIn by actively engaging with content related to cybersecurity.

### Skills Developed:

- LinkedIn Profile Optimization
- Networking and Professional Engagement
- Awareness of cybersecurity trends

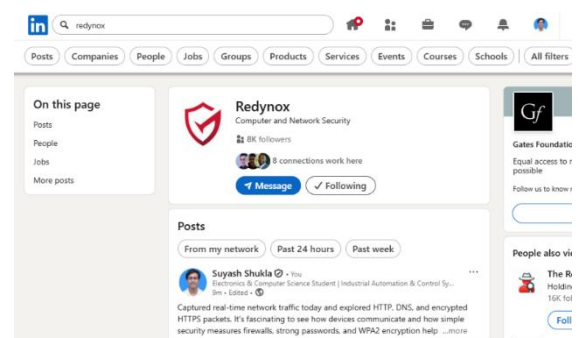
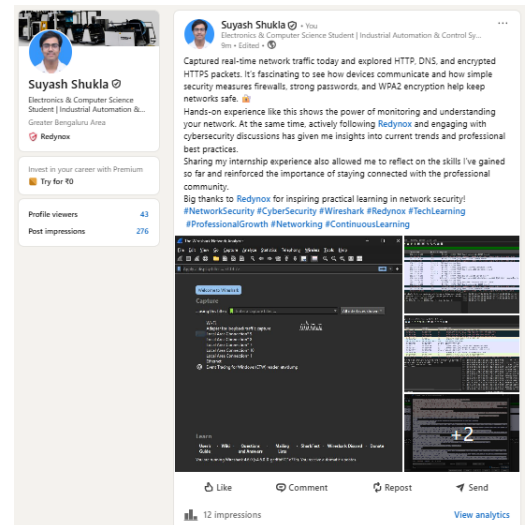
### Steps Taken:

#### 1. Followed Redynox on LinkedIn:

- Visited the official LinkedIn page of Redynox ([Redynox LinkedIn](#)).
- Clicked on **Follow** to stay updated with their posts and activities related to cybersecurity.
- My LinkedIn Profile [Clickme](#)

#### 2. Shared a Post About the Internship:

- Created a personalized post on my LinkedIn profile highlighting my experience and learnings during the internship.
- Emphasized the skills I gained and thanked Redynox for the opportunity.

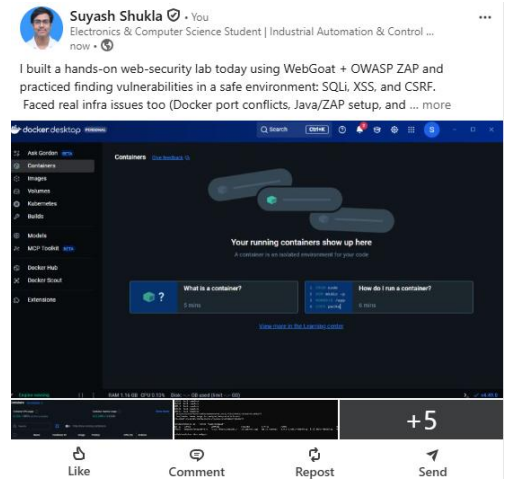




- Screenshot attached as proof.

### 3. Mentioned Redynox in a Post Discussing Cybersecurity:

- Wrote a LinkedIn post discussing the importance of cybersecurity in today's digital world.
- Mentioned Redynox to show association and engagement with industry-relevant companies.
- Screenshot attached as proof.



### 4. Provided Proof of Engagement:

Screenshots include:

- Redynox LinkedIn page showing the **Follow** button clicked.
- My shared post about the internship.
- My post discussing cybersecurity with Redynox mentioned.

### Outcome:

- Successfully enhanced LinkedIn presence and engagement with cybersecurity content.
- Demonstrated active networking by interacting with a professional organization in the field.

| A  |                                                  | B                                                                       |
|----|--------------------------------------------------|-------------------------------------------------------------------------|
| 1  | Post URL                                         | https://www.linkedin.com/feed/update/urn:li:ugcPost:7387772087190441984 |
| 2  | Post Date                                        | Oct 25, 2025                                                            |
| 3  | Post Publish Time                                | 8:48 AM                                                                 |
| 4  |                                                  |                                                                         |
| 5  | Post Performance                                 |                                                                         |
| 6  | Impressions                                      | 59                                                                      |
| 7  | Members reached                                  | 57                                                                      |
| 8  | Profile viewers from this post                   | 2                                                                       |
| 9  | Followers gained from this post                  | 0                                                                       |
| 10 | Reactions                                        | 7                                                                       |
| 11 | Comments                                         | 0                                                                       |
| 12 | Reposts                                          | 0                                                                       |
| 13 | Saves                                            | 0                                                                       |
| 14 | Sends on LinkedIn                                | 0                                                                       |
| 15 |                                                  |                                                                         |
| 16 | Reactions Highlights Oct 25, 2025 to Nov 1, 2025 |                                                                         |
| 17 | Top job title                                    | Computer Vision Engineer                                                |
| 18 | Top location                                     | Greater Bengaluru Area                                                  |
| 19 | Top industry                                     | IT Services and IT Consulting                                           |
| 20 |                                                  |                                                                         |